Class 6: Decision Support, APIs, and User Interfaces

Data-driven Systems Engineering

Agenda

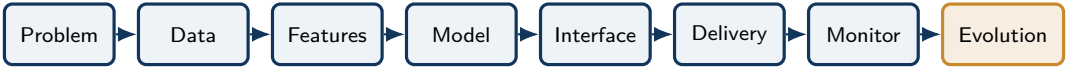
- From prediction to decision support.
- Why model interfaces matter.
- Flask as a service layer and Streamlit as a presentation layer.
- Repository examples for fraud detection and streaming dashboards.
- Risks: input validation, thresholds, logging, and trust.

Learning goals

Learning goals

- Distinguish a prediction from a decision.
- Explain the roles of APIs and dashboards in ML systems.
- Compare Flask and Streamlit in the context of this course.
- Identify the minimum quality requirements for a credible model interface.

Course Map and Running Cases



Today in the course

Focus: Interfaces that turn model outputs into usable decisions.
Bridge: This class connects modeling work to the first real system boundary: APIs, dashboards, and human-facing decision support.

Fraud case

In the fraud case, the model becomes a scoring endpoint that other services or analysts can call and interpret.

Pasteurization case

In the pasteurization case, the model becomes a streaming dashboard where sensor state, prediction, and alert behavior are visible together.

Course thread: Once a model has an interface, software engineering concerns become unavoidable rather than optional.

A model creates value only through an interface

- A trained model that nobody can call or interpret has limited practical value.
- Interfaces make predictions available to users, services, and operational workflows.
- The interface also shapes trust: input checks, response format, and clarity matter.

Prediction is not the final decision

- A fraud score suggests escalation, not guilt.
- A medical risk score supports a clinician, not a replacement for judgment.
- A drift alert suggests investigation, not automatic panic.

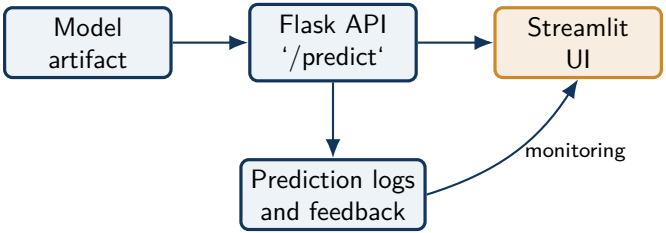
Engineering implication

Interfaces should communicate uncertainty, threshold logic, and next actions when possible.

Flask and Streamlit play different roles

Flask	Best when we need programmable endpoints over HTTP, such as ‘POST /predict‘ or a streaming route.
Streamlit	Best when we need a fast interactive UI for demos, dashboards, or exploratory interfaces.
Together	A common pattern is Flask for the backend service and Streamlit for the human-facing interface.

Repository flow



Concrete repository examples

Repo activity

- ‘Class6_model_api.py‘ exposes a basic prediction service through Flask.
- ‘streamlit/pages/App_Credit_Card.py‘ turns a fraud model into an interactive decision-support interface.
- ‘pasteurization/serving.py‘ exposes synthetic streaming sensor data through Flask.
- ‘streamlit/pages/App_Pasteurization.py‘ and ‘App_Stream.py‘ consume live data for visualization and monitoring.

Repository example: minimal Flask prediction endpoint

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(force=True)
    features = np.array(data["features"]).reshape(1, -1)
    pred = model.predict(features)[0]
    return jsonify({"prediction": int(pred)})
```

- This comes directly from ‘Class6_model_api.py‘.
- It is useful pedagogically because students can immediately see where input schema, validation, and logging are still missing.

Repository example: Streamlit decision-support interaction

```
if st.button("Predict Fraud Risk"):
    prediction = model.predict(scaled)[0]
    probability = model.predict_proba(scaled)[0][1]

    if prediction == 1:
        st.error(f"Fraud Probability: {probability:.2%}")
    else:
        st.success(f"Fraud Probability: {probability:.2%}")
```

- This mirrors ‘streamlit/pages/App_Credit_Card.py‘.
- The key difference from Flask is audience: a human user needs labels, context, and actionable feedback rather than raw JSON.

What a better API should include

- A clear input schema.
- Error handling for malformed or incomplete requests.
- Stable JSON output with prediction and confidence where appropriate.
- Logging hooks for later monitoring and debugging.
- Health or status endpoints for operational checks.

What a better dashboard should include

- A visible explanation of what the model does and does not do.
- Controls that reflect meaningful business inputs, not arbitrary technical internals.
- Results that support action: alert, score, trend, or explanation.
- Consistent links to data provenance, model version, or monitoring signals.

Streaming interface example: live learning loop

```
response = requests.get(URL)
data = response.json()["current_weather"]
temp = data["temperature"]
wind = data["windspeed"]

y_pred = model.predict_one({"wind": wind})
model.learn_one({"wind": wind}, temp)
adwin.update(abs(temp - (y_pred or 0)))
```

- ‘streamlit/pages/App_Stream.py‘ shows that some interfaces are not single-request forms but continuous monitoring surfaces.
- This also links directly to drift detection and operational dashboards later in the course.

Where trust breaks down

- The interface hides uncertainty.
- The inputs shown to users do not match training assumptions.
- The API accepts bad requests silently.
- The dashboard makes a model look authoritative when it is only a demo.

Technology links

Papers and materials

Flask documentation; Flask tutorial; Streamlit documentation; Get started with Streamlit.

From This Class to the Next

What stays with us

- Interfaces translate model behavior into decisions, workflows, and user trust.
- Flask and Streamlit solve related but different problems in system design.
- Serving requires validation, error handling, and clarity about uncertainty.

Project use

Students should now see how the same model can appear as an API for integration or as a dashboard for monitoring and decision support.

Next connection

Class 7 broadens the view from single applications to software engineering foundations: quality, architecture, maintainability, and lifecycle discipline.

Course thread: Once the service exists, the next question is whether it is engineered well enough to survive real use and future change.

Papers and materials

Papers and materials

Sculley et al., Hidden Technical Debt in Machine Learning Systems; The Twelve-Factor App; Designing Machine Learning Systems; Streamlit docs and examples.